

Table of Contents

Risk Documentation	2
Interactive House	2
Revision History.....	2
Risk List	2
Risk Handling Plans	3
R1: Database Service Downtime or Connectivity Issues	3
R2: Inconsistency in Real-Time Synchronization	3
R3: Misconfigured Security Rules or Unauthorized Access	3
R4: Vendor Lock-In & Technology Dependency	4
R5: Performance Degradation Under High Real-Time Load	4
R6: Gateway Availability Dependency	4
R7: Database Synchronization Failure	5
R8: No Backup Database	5
Risk Status Summary	5

Risk Documentation

Interactive House

Revision History

Date	Version	Description	Author
08/02/26	1.0	Initial Risk Assessment	Katerina Arvay, Dario Ostojic, Andre Sandblom
01/03/26	1.1	Update of Risk Assessment to reflect adoption of Firebase Firestore Database	Katerina Arvay, Dario Ostojic, Andre Sandblom
17/03/26	1.2	Update of Risk Assessment to reflect deployment setup and current system architecture (Firebase Firestore + gateway integration)	Katerina Arvay, Dario Ostojic, Andre Sandblom
23/4/26	1.3	Revised Firebase wording with dual database language and added Risk of Database Synchronization	Katerina Arvay, Dario Ostojic, Andre Sandblom
08/05/26	1.4	Risk management section added (likelihood/impact matrix); priorities updated; R8 added (no backup DB)	Katerina Arvay, Dario Ostojic, Andre Sandblom

Risk List

Risk Description	Priority
R1. Database Service Downtime or Connectivity Issues	Medium
R2. Inconsistency in Real-Time Synchronization	High
R3. Misconfigured Security Rules or Unauthorized Access	High
R4. Vendor Lock-In & Technology Dependency	Low
R5. Performance Degradation Under High Real-Time Load	Medium
R6. Gateway Availability Dependency	High
R7. Database Synchronization Failure	High
R8. No Backup Database (Supabase not implemented)	High

Risk Handling Plans

R1: Database Service Downtime or Connectivity Issues

Description: The Interactive House system relies on both primary and secondary database services as a cloud-based backend. If both primary and secondary database services become temporarily unavailable or if client devices lose internet connectivity, core functionality such as device control and monitoring may be disrupted.

Probability: Low

Impact: High

Mitigation Plan:

- Implement graceful error handling in the client application
- Cache critical device states locally where appropriate
- Automatically resynchronize when connectivity is restored
- Monitor Firebase service status during development and testing
- Utilize secondary database for recovery where applicable (currently not implemented — see R8)

R2: Inconsistency in Real-Time Synchronization

Description: Inconsistencies may arise between the system state stored in the primary database and the backup database, as well as between the stored state and the actual physical device state.

Probability: Medium

Impact: High

Mitigation Plan:

- Use Firebase transactions for critical updates
- Validate data before writing to the database
- Structure data hierarchically to reduce write conflicts
- Thoroughly test concurrent update scenarios
- Implement synchronization validation mechanisms
- Regularly verify consistency between databases

R3: Misconfigured Security Rules or Unauthorized Access

Description: Improperly configured access control or authentication mechanisms may allow unauthorized access to user data or device controls.

Probability: Medium

Impact: High

Mitigation Plan:

- Enforce Firebase authentication for all users
- Define and deploy strict Firebase Security Rules — URGENT: SG1 Firestore rules currently allow all reads/writes until Oct 2026; proper rules must be deployed before production use
- Test security rules with both valid and invalid access attempts
- Follow least privileged principles

R4: Vendor Lock-In & Technology Dependency

Description: Although the system uses multiple database technologies, dependencies on specific platforms may still introduce challenges when migrating or modifying the system architecture.

Probability: Medium

Impact: Medium

Mitigation Plan:

- Keep business logic separated from database-specific code
- Avoid overly complex Firebase-specific features where possible
- Document data model clearly to ease future migration if required
- Abstract both database interactions
- Keep DB-agnostic interfaces

R5: Performance Degradation Under High Real-Time Load

Description: Many simultaneous real-time listeners may lead to increased bandwidth usage and slower performance.

Probability: Low to Medium

Impact: Medium

Mitigation Plan:

- Structure database to minimize unnecessary listeners
- Use shallow queries where possible
- Monitor usage metrics during testing

R6: Gateway Availability Dependency

Description: The system relies on a locally hosted gateway to communicate with physical devices. If the gateway machine is offline, unreachable, or loses connectivity, command execution and real-time updates may be disrupted.

Probability: Medium

Impact: High

Mitigation Plan:

- Ensure the gateway runs on a stable, continuously powered device
- Implement health checks (e.g. /health endpoint)
- Provide fallback mechanisms or status indicators in the client
- Cloudflare Tunnel is implemented and provides a stable public endpoint. Risk elevated to High: the final demonstration depends on the locally hosted gateway. Pre-demo: verify machine powered, Cloudflare Tunnel active, /health endpoint responds.

R7: Database Synchronization Failure

Description: Failures in synchronization mechanisms between the primary and secondary databases may result in outdated, missing, or conflicting data across the system.

Probability: Medium

Impact: High

Mitigation Plan:

- Define clear synchronization strategy (e.g. event-based or periodic)
- Log synchronization operations for traceability
- Implement retry mechanisms for failed sync operations
- Regularly validate data consistency between databases. Note: secondary database not implemented (see R8), meaning Firestore data loss cannot be recovered — this elevates the effective impact of this risk.
-

R8: No Backup Database

Description: The planned secondary relational database (Supabase) has not been implemented. The supabase/ folder in the repository contains only placeholder stub code. The system relies entirely on Firebase Firestore. If Firestore data is lost or corrupted, there is no backup or recovery path.

Probability: Low

Impact: High

Priority: High

Mitigation Plan:

- Implement Supabase as secondary relational database (future work)
- Define event-based synchronisation strategy between Firestore and Supabase
- Implement retry mechanisms for failed synchronisation operations
- Until implemented: formally accept this risk; perform manual Firestore data export via Firebase Console before the final demonstration

Risk Status Summary

Risk	Description	Priority	Status	Key Mitigation
R1	Database Downtime	Medium	Mitigated	Firebase HA; graceful error handling
R2	Sync Inconsistency	High	Ongoing	Firestore transactions; two-collection arch
R3	Security Rules Open	High	Action Required	Deploy proper rules before demo
R4	Vendor Lock-In	Low	Accepted	Architecture locked; DB-agnostic interfaces
R5	Performance Degradation	Medium	Monitored	Shallow queries; minimal listeners
R6	Gateway Dependency	High	Action Required	Verify gateway and Cloudflare Tunnel pre-demo
R7	DB Sync Failure	High	Action Required	No backup DB yet; see R8
R8	No Backup Database	High	Accepted	Manual export before demo; Supabase future work